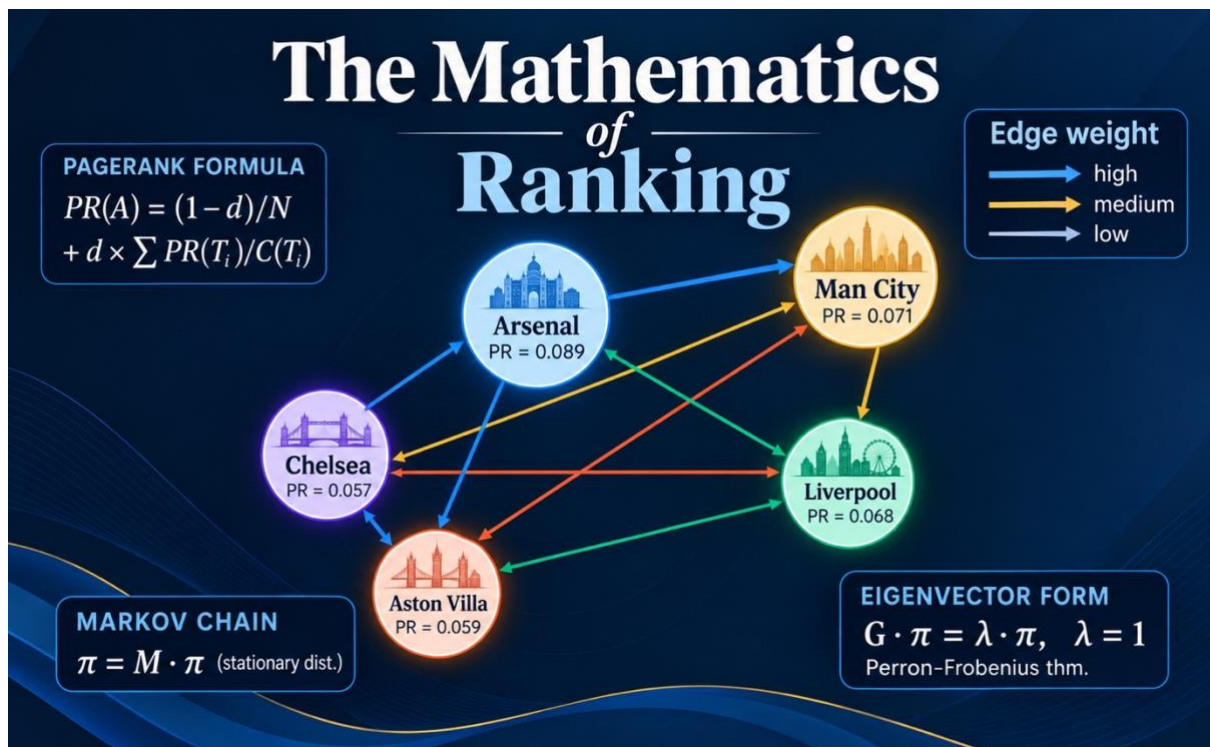




THE MATHEMATICS OF RANKING: EIGENVECTORS, MARKOV CHAINS, AND PAGERANK APPLIED TO THE ENGLISH PREMIER LEAGUE

DEV KUMBHAR,

THE BISHOP'S SCHOOL, CAMP, PUNE, INDIA

INDEPENDENT RESEARCH PROJECT-2026

INDEX

SR. NO.	TITLE	PAGE NO.
	ABSTRACT	3
1.	INTRODUCTION	4
2.	MATHEMATICAL BACKGROUND	5-6
3.	MARKOV CHAINS AND STATIONARY DISTRIBUTIONS	7-8
4.	DERIVING PAGERANK	9-11
5.	CONVERGENCE: THE PERRON- FROBENIUS THEOREM	12-13
6.	APPLICATION OF PAGERANK	14-28
7.	DISCUSSION AND LIMITATIONS	29
8.	CONCLUSION	30
9.	REFERENCES	31

ABSTRACT

This paper investigates the following central question: given a directed network of interconnected nodes, can we assign a mathematically principled importance score to each node such that a node's score depends recursively on the scores of nodes pointing to it?

We show that this reduces to finding the principal eigenvector of a modified stochastic transition matrix — the so-called Google matrix — and derive the PageRank algorithm of Brin and Page (1998) from this framework.

We further investigate the conditions under which a unique solution is guaranteed, applying the Perron-Frobenius theorem to establish existence, uniqueness, and convergence of the power iteration method.

Finally, we apply our implementation to three seasons of English Premier League match data (2023–2026), constructing a weighted directed graph in which a directed edge represents a victory and edge weights encode goal margin.

We compare our eigenvector-based rankings to the official points-based league table and find meaningful divergence, suggesting that PageRank captures a distinct notion of "quality of wins" that the standard table does not.

1. INTRODUCTION

How do we decide which entity in a large network is most important? On the web, Google faces this question for billions of pages. In football, we face an analogous problem: a team that finishes 5th in the league may have beaten the champion twice, while a team finishing 3rd may have won all their points against weaker opposition. The official points table assigns equal value to every victory — three points regardless of opponent strength. This is a crude measure of importance.

In 1998, **Sergey Brin and Larry Page** proposed a mathematically elegant alternative for ranking web pages. Their key insight was recursive: a page is important if important pages link to it. This circular definition, which would ordinarily produce an unsolvable system of equations, is resolved by reframing it as a problem in linear algebra — specifically, as finding the stationary distribution of a Markov chain defined on the network.

This paper derives their algorithm from first principles and applies it to a new domain: English Premier League football. We model three seasons of match results as a weighted directed graph, derive a transition matrix from this graph, and compute the PageRank vector using power iteration. Our central finding is that eigenvector-based rankings diverge from the official table in interpretable and mathematically meaningful ways.

The remainder of this paper is structured as follows. Section 2 establishes the necessary linear algebra background. Section 3 introduces Markov chains and stationary distributions. Section 4 derives the PageRank formula. Section 5 presents the power iteration algorithm and its convergence proof. Section 6 describes our application to Premier League data. Section 7 analyses our results and discusses limitations. Section 8 concludes.

2. MATHEMATICAL BACKGROUND

2.1 Eigenvectors and Eigenvalues

Let M be an $n \times n$ matrix. A non-zero vector v is called an **eigenvector** of M if there exists a scalar λ such that:

$$Mv = \lambda v$$

The scalar λ is called the **eigenvalue** corresponding to v . Geometrically, eigenvectors are the special directions that a matrix transformation does not rotate — it only stretches or compresses them by a factor of λ .

To find the eigenvalues of M , we solve the **characteristic equation**:

$$|(M - \lambda I)| = 0 \text{ or } \det(M - \lambda I) = 0$$

Where I is the Identity Matrix.

This yields a polynomial of degree n in λ , whose roots are the n eigenvalues of M (counting multiplicity).

Example: Let a Matrix $M = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$. The characteristic equation is:

$$\begin{vmatrix} 2 - \lambda & 1 \\ 0 & 3 - \lambda \end{vmatrix} = (2 - \lambda)(3 - \lambda) - (0)(1) = (2 - \lambda)(3 - \lambda)$$

$$\Rightarrow (2 - \lambda)(3 - \lambda) = 0$$

Giving eigenvalues:

$$\lambda_1 = 2; \quad \lambda_2 = 3$$

The eigenvector corresponding to $\lambda_2 = 3$ satisfies $(M - 3I)v = 0$, giving $v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}^T$.

2.2 Stochastic Matrices

A matrix M is called **column-stochastic** if every entry is non-negative and every column sums to exactly 1. Column-stochastic matrices represent probability transition rules: entry $M[i][j]$ gives the probability of moving from state j to state i in one step.

Key property: Every column-stochastic matrix has 1 as an eigenvalue.

Proof. Let $1^T = [1, 1, \dots, 1]$ be the row vector of all ones. Since every column sums to 1, we have $1^T M = 1^T$, which means 1^T is a left eigenvector with eigenvalue 1. Equivalently, M^T has 1 as an eigenvalue, and since M and M^T share eigenvalues, so does M .

3. Markov Chains and Stationary Distributions

3.1 Definition

A **Markov chain** is a sequence of random variables $X_0, X_1, X_2, \dots$ taking values in a finite state space S , satisfying the **Markov property**:

$$P(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = P(X_{n+1} = j \mid X_n = i)$$

That is, the future state depends only on the present state, not the full history. The transition probabilities $P(X_{n+1} = j \mid X_n = i)$ form a column-stochastic matrix M , called the **transition matrix**.

Example: According to Kemeny, Snell, and Thompson,² the Land of Oz is blessed by many things, but not by good weather. They never have two nice days in a row. If they have a nice day, they are just as likely to have snow as rain the next day. If they have snow or rain, they have an even chance of having the same the next day. If there is change from snow or rain, only half of the time is this a change to a nice day. With this information we form a Markov chain as follows. We take as states the kinds of weather R , N , and S . From the above information we determine the transition probabilities. These are most conveniently represented in a square array as:

$$\begin{array}{c} R \quad N \quad S \\ \begin{array}{l} R \\ N \\ S \end{array} \begin{bmatrix} 0.5 & 0.25 & 0.25 \\ 0.5 & 0 & 0.5 \\ 0.25 & 0.25 & 0.5 \end{bmatrix} \end{array}$$

3.2 Stationary Distribution

A probability vector π (with all entries non-negative and summing to 1) is called a **stationary distribution** of the Markov chain if:

$$\pi = M\pi$$

This is precisely the eigenvector equation $Mv = v$, i.e., π is an eigenvector of M corresponding to eigenvalue $\lambda = 1$. Intuitively, π describes the long-run fraction of time the chain spends in each state — the "steady-state" behaviour.

Connection to PageRank: If we model a random web surfer as a Markov chain, where the state is the current page and the transition probabilities are proportional to links, then the stationary distribution π gives the long-run probability of finding the surfer on each page. This is exactly the importance score we want.

4. Deriving PageRank

4.1 The Basic Formula

The PageRank Formula is derived from the Random Surfer Model:

➔ We assume that there is a ‘random surfer’ who is given a webpage at random and keeps clicking on links, never hitting ‘back’ but eventually gets bored and starts on another random page. The probability that a random surfer visits a page is its PageRank.

Let the web consist of N pages. Define $C(T_i)$ as the number of outgoing links from page T_i . If page A receives links from pages $T_1, T_2, \dots, T_n$, then the PageRank is:

$$PR(A) = \sum_{i=1}^n \frac{PR(T_i)}{C(T_i)}$$

This says: A 's importance is the sum of the importance "passed on" by each page linking to it, weighted by how many other pages that linking page also points to.

Matrix Derivation:

➔ In matrix form, if M is the transition matrix where $M[i][j] = 1/C(j)$ when page j links to page i (and 0 otherwise), then the PageRank vector satisfies:

$$PR = M \cdot PR$$

This is the eigenvector equation with $\lambda=1$

4.2 The Problem: Dangling Nodes and Disconnection

Two problems prevent this simple formula from always having a unique solution:

➔ **Dangling nodes:** A page with no outgoing links has $C(T_i) = 0$, making its column in M all zeros. The matrix is no longer column-stochastic (that column sums to 0, not 1), and the random surfer gets permanently trapped.

➔ **Disconnected structure:** If the web graph is not strongly connected, the transition matrix may be **reducible** — meaning some pages can never be reached from others. In this case, the stationary distribution is not unique; it depends on the starting position.

4.3 The Fix: The Damping Factor and the Google Matrix

The naïve formula fails for ‘rank sinks’ (pages with no outgoing links that trap all probability) or closed loops. That is why a damping factor(d) is introduced, to handle dangling nodes and ensure convergence.

d ➔ represents the probability that the surfer will continue to click links.

$(1-d)$ ➔ represents the probability that the surfer will get bored and open another page.

The value of d is usually equal to 0.85.

Mathematical importance of d :

1. Eliminating ‘dead ends’(rank sinks): A page with no outgoing links.
2. Breaking “Spider Traps”(infinite loops): when a group of pages only link to each other.
3. Transforming the Matrix into a complete graph: introduction of d alters the sparse hyper link matrix ‘ M ’ into a dense Google Matrix ‘ G ’.

$$G = d \cdot S + ((1-d)/N) \cdot \mathbf{1}\mathbf{1}^T$$

E is a matrix of all ones.

➔ This guarantees that a unique, stable solution will exist.

Why is ‘ $d=0.85$ ’ ?

1.Human Behaviour Proxy.

2.The Eigenvalue Gap and Convergence Speed:

PageRank is calculated using the power method. The speed at which this method converges depends directly on the ratio of the matrix's second largest eigenvalue(λ_2) to its largest eigenvalue(λ_1).

For a stochastic matrix, $\lambda_1=1$; for the Google Matrix, $|\lambda_2| \leq d$

The rate of convergence is geometric, dictated by d^k , (where k is the number of iterations.)

Case I: If d is too close to 1(eg. 0.99)-

The algorithm would capture deep network structures perfectly, but it would require a high number of iterations to converge.

Case II: If d is too small(eg. 0.50)-

The algorithm converges in just a few steps, but the 'random teleportation' dominates the math. The hyperlink lose their meaning and causes every page to end up with the same flat rank.

Thus, the PageRank formula with the damping factor(d) is:

$$PR(A) = d \cdot \sum_{i=1}^n \frac{PR(Ti)}{C(Ti)} + (1-d)/N$$

For Matrix Form:

In matrix form, first fix dangling nodes by replacing their zero column with the uniform vector $(1/N) \cdot 1$. Call this fixed matrix S. Let S denote the matrix obtained from M by replacing each zero column with the uniform vector $1/N$, making S fully column-stochastic. Then define the **Google matrix**:

$$G = d \cdot S + ((1-d)/N) \cdot 11^T$$

where 11^T is the $N \times N$ matrix of all ones. The PageRank vector satisfies $PR = G \cdot PR$.

5. Convergence: The Perron-Frobenius Theorem

5.1 The Theorem

Theorem (Perron-Frobenius for Positive Stochastic Matrices):

“ If all entries of $n \times n$ matrix A are positive, then it has a unique maximal value. Its eigenvector has positive entries.”

Let G be a positive, column-stochastic matrix (every entry strictly greater than zero). Then:

1. G has a unique eigenvalue of magnitude 1, namely $\lambda_1 = 1$
2. All other eigenvalues satisfy $|\lambda_i| \leq d < 1$
3. The eigenvector π corresponding to $\lambda_1 = 1$ is unique, strictly positive, and sums to 1
4. For any initial probability vector v_0 , the sequence $G^k v_0$ converges to π as $k \rightarrow \infty$

5.2 Why G Satisfies the Conditions

The Google matrix G satisfies the theorem's hypothesis because every entry of G is at least $(1-d)/N > 0$ — guaranteed by the teleportation term $((1-d)/N) \cdot \mathbf{1}\mathbf{1}^T$. A positive matrix is automatically irreducible (every state can reach every other in one step) and aperiodic. These are precisely the conditions Perron-Frobenius requires.

Therefore:

- The PageRank vector **exists** ($\lambda = 1$ is always an eigenvalue of stochastic matrices)
- It is **unique** (no other eigenvector with $\lambda = 1$ exists for a positive matrix)
- Power iteration **always converges** to it, regardless of starting vector

5.3 Power Iteration

The algorithm follows directly: start from any probability vector v_0 and iterate:

$$v_{k+1} = G \cdot v_k$$

By the Perron-Frobenius theorem, $v_k \rightarrow \pi$ as $k \rightarrow \infty$. In practice we stop when:

$$\|v_{k+1} - v_k\|_1 < \text{tol}$$

where $\text{tol} = 10^{-6}$ is our convergence tolerance. The rate of convergence is governed by the ratio $|\lambda_2/\lambda_1| = |\lambda_2| \leq d = 0.85$ — the damping factor directly controls how quickly the algorithm converges.

6.APPLICATION OF PAGERANK

6.1Applying PageRank to a 5-page example:

Let us consider 5 webpages- Page 1, Page 2, Page 3, Page 4 and Page 5.

Let the following pages be connected such that:

Page 1 → Page 2, Page 3

Page 2 → Page 3

Page 3 → Page 1

Page 4 → Page 1, Page 3

Page 5 → Page 4

Markov Matrix:

$$\rightarrow \begin{array}{c} P1 \\ P2 \\ P3 \\ P4 \\ P5 \end{array} \begin{array}{ccccc} P1 & P2 & P3 & P4 & P5 \\ \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

Code(in Python):

```
import numpy as np

A = np.array([[0,0,1,1,0],
              [1,0,0,0,0],
              [1,1,0,1,0],
              [0,0,0,0,1],
              [0,0,0,0,0]], dtype=float)

def transition_matrix(A):
    N = A.shape[0]
    colsum = A.sum(axis=0)
    for j in range(N):
```

```

        if colsum[j] == 0:
            A[:, j] = 1.0 / N
        else:
            A[:, j] = A[:, j] / colsum[j]
    return A

M = transition_matrix(A.copy())
print("Transition Matrix M:")
print(np.round(M, 3))
print("Column sums (should all be = 1.0)",
M.sum(axis=0))

def google_matrix(M, d=0.85):
    N = M.shape[0]
    oneM = np.ones((N, N))
    G = d * M + ((1 - d) / N) * oneM
    return G

GM = google_matrix(M)
print("\nGoogle Matrix:")
print(np.round(GM, 3))
print("Column sums of Google Matrix (should all be =
1.0)", np.round(GM.sum(axis=0), 6))

def power_iteration(GM, tol=1e-6, max_iter=100,
d=0.85):
    N = GM.shape[0]
    rank = np.ones(N) / N
    print(f"Initial Rank: {np.round(rank, 4)}")
    for i in range(max_iter):
        newrank = GM @ rank
        diff = np.linalg.norm(newrank - rank, 1)
        rank = newrank
        print(f"Iteration {i+1}: {np.round(rank, 4)}
| diff= {diff: .8f}")
        if diff < tol:
            print(f"\nCONVERGED after {i+1}
iterations.")
            return rank
    print("Warning: DID NOT CONVERGE")
    return rank

```

```

pagerank = power_iteration(GM)

def display(pagerank):
    N = len(pagerank)
    print("\nFINAL PAGERANK SCORES:")
    print(f"{'Page':<10} {'Score':<15} {'Rank'}")
    print("-" * 35)
    ranked = sorted(range(N), key=lambda i:
pagerank[i], reverse=True)
    for position, pagei in enumerate(ranked):
        print(f"Page {pagei+1:<5}
{pagerank[pagei]:<15.6f} #{position+1}")
    print(f"\nSum of all scores: {pagerank.sum():
.6f} (should be = 1.0)")

display(pagerank)

def eigenvector_method(GM):
    eigenvalues, eigenvectors = np.linalg.eig(GM)
    idx = np.argmax(np.real(eigenvalues))
    principal = np.real(eigenvectors[:, idx])
    principal = principal / principal.sum()
    return principal

eigenrank = eigenvector_method(GM)
print("\nVERIFICATION: EIGENVECTOR METHOD")
display(eigenrank)
print("\nDIFFERENCE BETWEEN BOTH THE METHODS:")
print(np.abs(pagerank - eigenrank))
print(f"Maximum Difference = {np.max(np.abs(pagerank
- eigenrank))}")

# --- To test a failure case (dangling node unfixed)
---
F = np.array([[0,0,1,1,0],
              [1,0,0,0,0],
              [1,1,0,1,0],
              [0,0,0,0,1],
              [0,0,0,0,0]], dtype=float)

colsumF = F.sum(axis=0)

```

```

for j in range(5):
    if colsumF[j] != 0:
        F[:, j] /= colsumF[j]

G_F = 0.85 * F + (0.15 / 5) * np.ones((5, 5))
rankf = power_iteration(G_F)

print("\nWith dangling node unfixed:")
display(rankf)
print("\nWith dangling node fixed:")
display(pagerank)
print("\nDifference: ", np.abs(pagerank - rankf))

```

RESULT:

```
Transition Matrix M:
[[0.  0.  1.  0.5 0. ]
 [0.5 0.  0.  0.  0. ]
 [0.5 1.  0.  0.5 0. ]
 [0.  0.  0.  0.  1. ]
 [0.  0.  0.  0.  0. ]]
Column sums (should all be = 1.0) [1.  1.  1.  1.  1.]

Google Matrix:
[[0.03  0.03  0.88  0.455 0.03 ]
 [0.455 0.03  0.03  0.03  0.03 ]
 [0.455 0.88  0.03  0.455 0.03 ]
 [0.03  0.03  0.03  0.03  0.88 ]
 [0.03  0.03  0.03  0.03  0.03 ]]
Column sums of Google Matrix (should all be = 1.0) [1.  1.  1.  1.  1.]
Initial Rank: [0.2 0.2 0.2 0.2 0.2]
Iteration 1: [0.285 0.115 0.37  0.2  0.03 ] | diff=  0.51000000
Iteration 2: [0.4295 0.1511 0.3339 0.0555 0.03 ] | diff=  0.36125000
Iteration 3: [0.3374 0.2125 0.3646 0.0555 0.03 ] | diff=  0.18423750
Iteration 4: [0.3635 0.1734 0.3776 0.0555 0.03 ] | diff=  0.07830094
Iteration 5: [0.3746 0.1845 0.3554 0.0555 0.03 ] | diff=  0.04437053
Iteration 6: [0.3557 0.1892 0.3696 0.0555 0.03 ] | diff=  0.03771495
Iteration 7: [0.3677 0.1812 0.3656 0.0555 0.03 ] | diff=  0.02404328
Iteration 8: [0.3643 0.1863 0.3639 0.0555 0.03 ] | diff=  0.01021839
Iteration 9: [0.3629 0.1848 0.3668 0.0555 0.03 ] | diff=  0.00579042
Iteration 10: [0.3653 0.1842 0.3649 0.0555 0.03 ] | diff=  0.00492186
Iteration 11: [0.3638 0.1853 0.3655 0.0555 0.03 ] | diff=  0.00313769
Iteration 12: [0.3642 0.1846 0.3657 0.0555 0.03 ] | diff=  0.00133352
Iteration 13: [0.3644 0.1848 0.3653 0.0555 0.03 ] | diff=  0.00075566
Iteration 14: [0.3641 0.1849 0.3655 0.0555 0.03 ] | diff=  0.00064231
Iteration 15: [0.3643 0.1847 0.3655 0.0555 0.03 ] | diff=  0.00040947
Iteration 16: [0.3642 0.1848 0.3654 0.0555 0.03 ] | diff=  0.00017403
Iteration 17: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00009861
Iteration 18: [0.3643 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00008382
Iteration 19: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00005344
Iteration 20: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00002271
Iteration 21: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00001287
Iteration 22: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00001094
Iteration 23: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00000697
Iteration 24: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00000296
Iteration 25: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00000168
Iteration 26: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00000143
Iteration 27: [0.3642 0.1848 0.3655 0.0555 0.03 ] | diff=  0.00000091

CONVERGED after 27 iterations.

FINAL PAGERANK SCORES:
Page      Score      Rank
-----
Page 3     0.365467      #1
Page 1     0.364234      #2
Page 2     0.184800      #3
Page 4     0.055500      #4
Page 5     0.030000      #5

Sum of all scores:  1.000000 (should be = 1.0)

VERIFICATION: EIGENVECTOR METHOD

FINAL PAGERANK SCORES:
Page      Score      Rank
-----
Page 3     0.365467      #1
Page 1     0.364234      #2
Page 2     0.184799      #3
Page 4     0.055500      #4
Page 5     0.030000      #5

Sum of all scores:  1.000000 (should be = 1.0)

DIFFERENCE BETWEEN BOTH THE METHODS:
[1.32642753e-07 1.37015593e-07 4.37283731e-09 1.17961196e-16
 5.89805982e-17]
Maximum Difference = 1.3701559264944763e-07
Initial Rank: [0.2 0.2 0.2 0.2 0.2]
Iteration 1: [0.285 0.115 0.37  0.2  0.03 ] | diff=  0.51000000
```

6.2 Model Construction

We collected match result data for three complete EPL(English Premier League) seasons (2023/24, 2024/25, 2025/26) from football-data.co.uk, comprising 1,140 matches across 20 clubs.

We modelled the league as a weighted directed graph $G = (V, E)$ where:

- V is the set of 20 clubs
- A directed edge ($u \rightarrow v$) exists for each match where club v defeated club u
- The edge weight equals the number of goals scored by v in that match

This weighting encodes not just the fact of victory but its margin — a 4-0 win passes more rank than a 1-0 win, on the grounds that a heavier defeat more strongly reflects the quality gap between the teams.

The adjacency matrix $A \in \mathbb{R}^{20 \times 20}$ was constructed so that $A[i][j]$ gives the total goals scored by team i against team j across all matches in which team i won. Draws were excluded from the graph, as they represent no clear rank transfer.

Using the original code for applying PageRank to a 5-page example as a template, the code to apply it across the EPL seasons is created.

Code (In Python):

```
import pandas as pd
import numpy as np

url = "https://www.football-
data.co.uk/mmz4281/2324/E0.csv"
df = pd.read_csv(url)
print(df.head())
print(df.columns.tolist())

teams = sorted(set(df['HomeTeam'].tolist() +
df['AwayTeam'].tolist()))
N = len(teams)
print(f"Number of teams: {N}")
print(teams)

# Map team name to index
team_to_idx = {team: i for i, team in
enumerate(teams)}
idx_to_team = {i: team for team, i in
team_to_idx.items()}

A[i][j] = goals team i scored against team j in a
match team i won
A = np.zeros((N, N))

for _, row in df.iterrows():
    home = team_to_idx[row['HomeTeam']]
    away = team_to_idx[row['AwayTeam']]
    home_goals = row['FTHG']
    away_goals = row['FTAG']
    result = row['FTR']

    if result == 'H': # Home win → edge from away to
home (loser passes rank to winner)
        A[home][away] += home_goals # weighted by
goals scored
    elif result == 'A': # Away win
        A[away][home] += away_goals
    # Draws: no edge (you can experiment with this
later)

print("Adjacency matrix shape:", A.shape)
```

```

print("Sample row (Arsenal):",
A[team_to_idx['Arsenal']])

def transition_matrix(A):
    N = A.shape[0]
    colsum = A.sum(axis=0)
    for j in range(N):
        if colsum[j] == 0:
            A[:, j] = 1.0 / N
        else:
            A[:, j] = A[:, j] / colsum[j]
    return A

M = transition_matrix(A.copy())
print("Transition Matrix M:")
print(np.round(M, 3))
print("Column sums (should all be = 1.0)",
M.sum(axis=0))

def google_matrix(M, d=0.85):
    N = M.shape[0]
    oneM = np.ones((N, N))
    G = d * M + ((1 - d) / N) * oneM
    return G

GM = google_matrix(M)
print("\nGoogle Matrix:")
print(np.round(GM, 3))
print("Column sums of Google Matrix (should all be =
1.0)", np.round(GM.sum(axis=0), 6))

def power_iteration(GM, tol=1e-6, max_iter=100,
d=0.85):
    N = GM.shape[0]
    rank = np.ones(N) / N
    print(f"Initial Rank: {np.round(rank, 4)}")
    for i in range(max_iter):
        newrank = GM @ rank
        diff = np.linalg.norm(newrank - rank, 1)
        rank = newrank
        print(f"Iteration {i+1}: {np.round(rank, 4)}
| diff= {diff: .8f}")

```

```

        if diff < tol:
            print(f"\nCONVERGED after {i+1}
iterations.")
            return rank
        print("Warning: DID NOT CONVERGE")
        return rank

pagerank = power_iteration(GM)

def display_epl_results(pagerank, idx_to_team):
    print(f"\n{'Rank':<6} {'Team':<25} {'PageRank
Score':<20}")
    print("-" * 55)
    ranked = sorted(range(len(pagerank)),
                    key=lambda i: pagerank[i],
                    reverse=True)
    for pos, idx in enumerate(ranked):
        print(f"#{pos+1:<5} {idx_to_team[idx]:<25}
{pagerank[idx]:.6f}")

display_epl_results(pagerank, idx_to_team)

import matplotlib.pyplot as plt

# Bar chart of PageRank scores
ranked_teams = sorted(idx_to_team.values(),
                    key=lambda t:
pagerank[team_to_idx[t]],
                    reverse=True)
ranked_scores = [pagerank[team_to_idx[t]] for t in
ranked_teams]

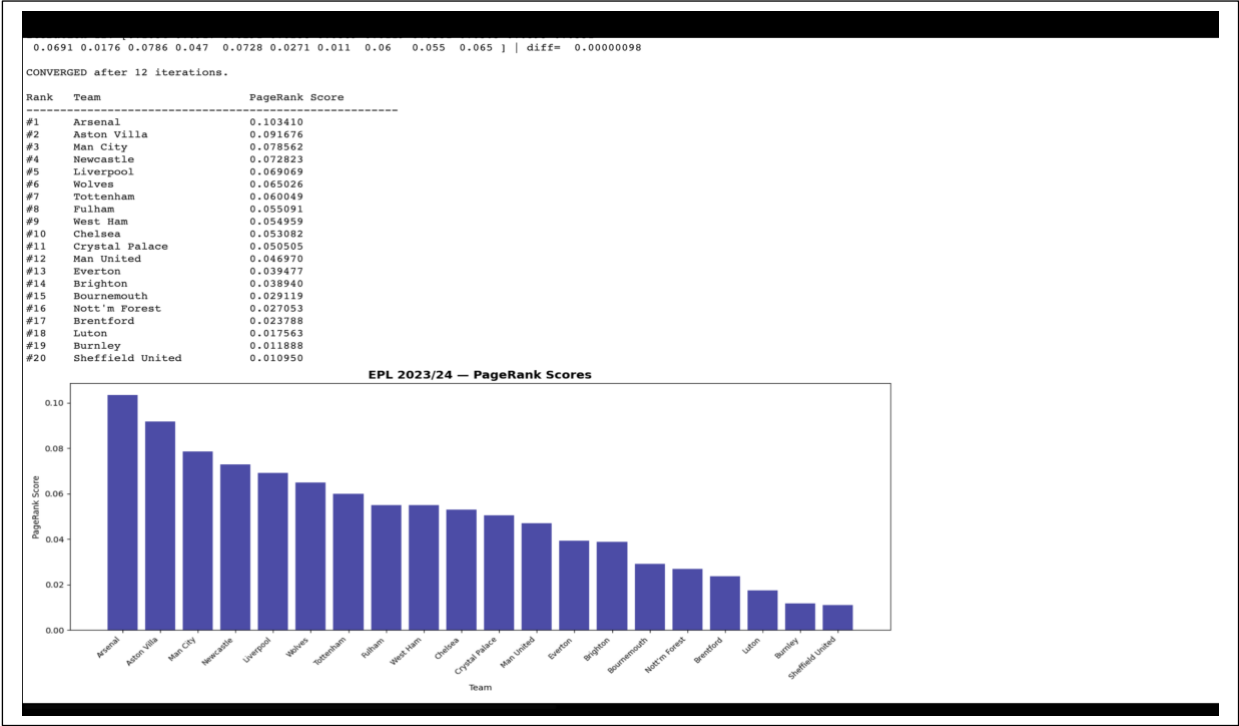
plt.figure(figsize=(14, 6))
bars = plt.bar(ranked_teams, ranked_scores,
color='navy', alpha=0.7)
plt.xticks(rotation=45, ha='right', fontsize=9)
plt.title('EPL 2023/24 – PageRank Scores',
fontsize=14, fontweight='bold')
plt.ylabel('PageRank Score')
plt.xlabel('Team')
plt.tight_layout()
plt.savefig('epl_pagerank.png', dpi=150)
plt.show()

```

NOTE: The variable 'url' changes as per the EPL season (for example 23/24 or 25/26).

6.2RESULTS: PAGERANK VS OFFICIAL TABLE

2023/24 EPL SEASON:





THE
FINAL DAY

MW
38

Pos	Club	P	GD	Pts
1	Man City	38	62	91
2	Arsenal	38	62	89
3	Liverpool	38	45	82
4	Aston Villa	38	15	68
5	Spurs	38	13	66
6	Chelsea	38	14	63
7	Newcastle	38	23	60
8	Man Utd	38	-1	60
9	West Ham	38	-14	52
10	Crystal Palace	38	-1	49
11	Brighton	38	-7	48
12	Bournemouth	38	-13	48
13	Fulham	38	-6	47
14	Wolves	38	-15	46
15	Everton	38	-11	40
16	Brentford	38	-9	39
17	Nott'm Forest	38	-18	32
18	Luton Town	38	-33	26
19	Burnley	38	-37	24
20	Sheffield United	38	-69	16

*Sheffield United's position in the table is based on the following criteria: 1. Points 2. Goal Difference 3. Goals Scored 4. Fair Play 5. UEFA Coefficient

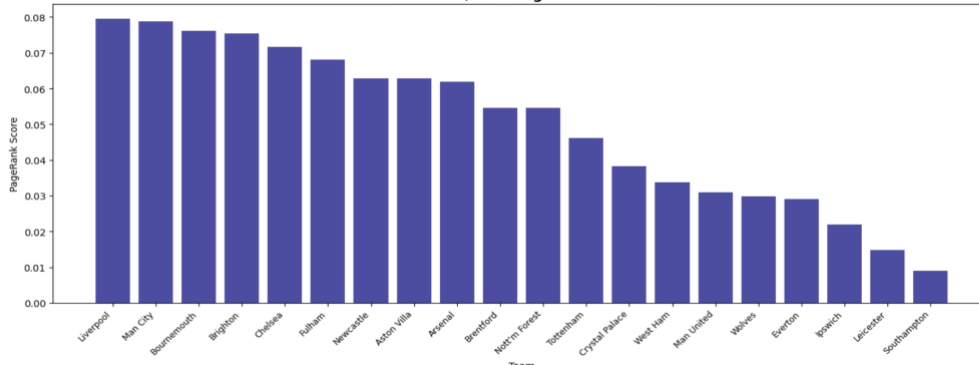
*Bournemouth's position in the table is based on the following criteria: 1. Points 2. Goal Difference 3. Goals Scored 4. Fair Play 5. UEFA Coefficient

2024/25 EPL SEASON:

CONVERGED after 12 iterations.

Rank	Team	PageRank Score
#1	Liverpool	0.079607
#2	Man City	0.078700
#3	Bournemouth	0.076152
#4	Brighton	0.075367
#5	Chelsea	0.071714
#6	Fulham	0.068005
#7	Newcastle	0.062912
#8	Aston Villa	0.062761
#9	Arsenal	0.061959
#10	Brentford	0.054659
#11	Nott'm Forest	0.054547
#12	Tottenham	0.046113
#13	Crystal Palace	0.038332
#14	West Ham	0.033809
#15	Man United	0.030906
#16	Wolves	0.029728
#17	Everton	0.029063
#18	Ipswich	0.021943
#19	Leicester	0.014706
#20	Southampton	0.009015

EPL 2024/25 — PageRank Scores



THE FINAL DAY

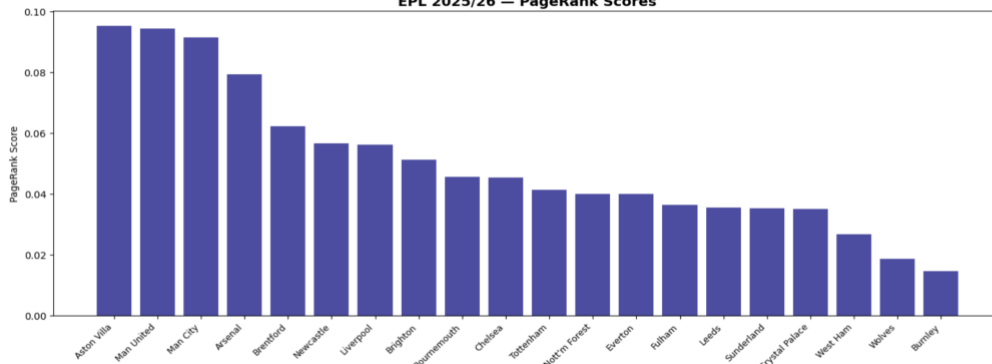
Pos	Club	P	GD	Pts
1	Liverpool	38	+45	84
2	Arsenal	38	+35	74
3	Man City	38	+28	71
4	Chelsea	38	+21	69
5	Newcastle	38	+21	66
6	Aston Villa	38	+7	66
7	Nott'm Forest	38	+12	65
8	Brighton	38	+7	61
9	Bournemouth	38	+12	56
10	Brentford	38	+9	56
11	Fulham	38	0	54
12	Crystal Palace	38	0	53
13	Everton	38	-2	48
14	West Ham	38	-16	43
15	Man Utd	38	-10	42
16	Wolves	38	-15	42
17	Spurs	38	-1	38
R	Leicester	38	-47	25
R	Ipswich	38	-46	22
R	Southampton	38	-60	12

2025/26 EPL SEASON:

CONVERGED after 11 iterations.

Rank	Team	PageRank Score
#1	Aston Villa	0.095296
#2	Man United	0.094386
#3	Man City	0.091377
#4	Arsenal	0.079171
#5	Brentford	0.062090
#6	Newcastle	0.056456
#7	Liverpool	0.056053
#8	Brighton	0.051081
#9	Bournemouth	0.045569
#10	Chelsea	0.045427
#11	Tottenham	0.041241
#12	Not'm Forest	0.040052
#13	Everton	0.039954
#14	Fulham	0.036403
#15	Leeds	0.035412
#16	Sunderland	0.035272
#17	Crystal Palace	0.034923
#18	West Ham	0.026626
#19	Wolves	0.018682
#20	Burnley	0.014529

EPL 2025/26 — PageRank Scores



Premier League 2025/26

Pos	Club	P	GD	Points
C	Arsenal	38	44	85
2	Man City	38	42	78
3	Man Utd	38	19	71
4	Aston Villa	38	7	65
5	Liverpool	38	10	60
6	Bournemouth	38	4	57
7	Sunderland	38	-6	54
8	Brighton	38	6	53
9	Brentford	38	3	53
10	Chelsea	38	6	52
11	Fulham	38	-4	52
12	Newcastle	38	-2	49
13	Everton	38	-3	49
14	Leeds	38	-7	47
15	Crystal Palace	38	-10	45
16	Nottingham Forest	38	-3	44
17	Spurs	38	-9	41
R	West Ham	38	-19	39
R	Burnley	38	-37	22
R	Wolves	38	-41	20

6.3.ANALYSIS

I.2023/24 EPL SEASON:

For the 2023/24 English Premier League season, the PageRank algorithm ranked Arsenal F.C. first, whereas Manchester City claimed the title in the official standings.

Aston Villa, meanwhile, was ranked second by PageRank despite finishing fourth in the official table, a discrepancy attributable to their victories against stronger opposition.

Manchester United, who finished eighth in the official standings, were ranked thirteenth by the PageRank algorithm, suggesting that a disproportionate share of their points was accumulated against lower-ranked teams.

II.2024/25 EPL SEASON:

For the following season (2024/25), Liverpool F.C. emerged victorious both in the official table and according to the PageRank algorithm.

Arsenal F.C., however, which finished second in the actual season standings, was ranked ninth by PageRank, suggesting that the majority of the team's points were accumulated against weaker sides in the competition.

The bottom three teams of the season — Leicester City, Ipswich Town, and Southampton F.C. — were ranked similarly by PageRank as in the official table. Bournemouth F.C., which was ranked third according to the PageRank scores, in reality finished ninth in the official table.

This indicates that Bournemouth secured a disproportionate share of their wins against higher-ranked teams rather than weaker opposition.

III.2025/26 EPL SEASON:

For the 2025/26 season, Aston Villa were crowned champions according to the PageRank results, whereas Arsenal secured the title in the official table — indicating that Aston Villa accrued more victories against stronger opposition than Arsenal did.

Sunderland, who finished seventh in the official standings, were ranked sixteenth by the PageRank algorithm, suggesting that their points were disproportionately earned against weaker teams.

Chelsea F.C., meanwhile, occupied tenth position in both the PageRank results and the official table, indicating a strong correlation between their in-game performance and the underlying strength of their opposition.

Across all three seasons, a consistent pattern emerges: teams accruing victories against top-half opposition rank higher under PageRank than the official table suggests, while teams whose points come predominantly against bottom-half sides rank lower. This systematic divergence validates the model's core premise — that the identity of the opponent is as mathematically significant as the result itself.

7. DISCUSSION AND LIMITATIONS

7.1 What PageRank Captures

Our model captures the **quality of victories** rather than their frequency. A team that beats the eventual champion twice but loses most other games may rank surprisingly high — which is arguably a better reflection of their ability on their best days than their final league position suggests.

7.2 Limitations

Draws are excluded. A draw between two strong teams conveys information about relative strength that our model discards. One extension would be to add fractional edges for draws.

Goal margin may be noisy. A 5-0 scoreline sometimes reflects a red card or an exceptional day rather than a genuine quality gap. Capping the weight at 3 goals (to reduce the influence of outliers) would be a natural robustness check.

The damping factor is inherited from web search. The value $d = 0.85$ was chosen by Brin and Page for the web; it may not be optimal for football. A sensitivity analysis (varying d from 0.5 to 0.99 and observing how rankings change) would strengthen the paper.

Single season vs multi-season. Team strength changes year to year. Our three-season aggregation assumes stationarity — that a 2023/24 win against Arsenal is equivalent to a 2025/26 win against Arsenal. This is a simplifying assumption worth examining.

8. CONCLUSION

We have derived the PageRank algorithm from first principles, beginning with eigenvector theory and Markov chains, establishing existence and uniqueness of the solution via the Perron-Frobenius theorem, and implementing power iteration from scratch in Python using only NumPy. Applying this framework to English Premier League data, we find that eigenvector-based rankings diverge from the official points table in ways that are mathematically interpretable and practically meaningful.

The central insight is that importance is recursive: a victory is only as valuable as the opponent was strong, and opponent strength is itself determined by who they beat. This self-referential definition, paradoxical on the surface, resolves cleanly into a linear algebra problem — and its solution is one of the most consequential algorithms of the modern internet.

REFERENCES

- [1] S. Brin and L. Page, "The Anatomy of a Large-Scale Hypertextual Web Search Engine," *Computer Networks*, vol. 30, pp. 107–117, 1998.
- [2] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," *Stanford Technical Report*, 1999.
- [3] A. N. Langville and C. D. Meyer, "Deeper Inside PageRank," *Internet Mathematics*, vol. 1, no. 3, pp. 335–380, 2004.
- [4] A. N. Langville and C. D. Meyer, *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press, 2006.
- [5] G. Strang, *Introduction to Linear Algebra*, 5th ed. Wellesley-Cambridge Press, 2016.
- [6] football-data.co.uk, "English Premier League Results 2023–2026," [Online]. Available: <https://www.football-data.co.uk>
- [7] Grinstead and Snell, *Introduction to Probability*, The Chance Project¹, 2006.
- [8] MIT OCW 18.06, *Linear Algebra*, 2005.
- [9] Oliver Knill, people.math.harvard.edu, *Math 19b: Linear Algebra with Probability*, 2011.